

Cahier des charges - Loc-Manager

Page de couverture

Titre : Cahier des charges - Loc-Manager

Auteur : [Votre nom]

Date : 25 mars 2025

Projet : Application web de gestion de location de véhicules

Version : 1.0

Tableau des matières

- [1. Présentation du projet](#)
 - [1.1 Contexte](#)
 - [1.2 Objectifs](#)
 - [1.3 Public cible](#)
- [2. Spécifications fonctionnelles](#)
 - [2.1 Fonctionnalités principales](#)
 - [2.2 Fonctionnalités secondaires](#)
- [3. Spécifications techniques](#)
 - [3.1 Architecture](#)
 - [3.2 Technologies utilisées](#)
 - [3.3 Structure des données \(Firestore\)](#)
 - [3.4 Contraintes techniques](#)
- [4. Exigences non fonctionnelles](#)
 - [4.1 Ergonomie](#)
 - [4.2 Sécurité](#)
 - [4.3 Disponibilité](#)
- [5. Livrables](#)
 - [5.1 Livrables principaux](#)
 - [5.2 Livrables secondaires](#)
- [6. Planning et jalons](#)
 - [6.1 Phases du projet](#)
 - [6.2 Jalons](#)
- [7. Contraintes et hypothèses](#)
 - [7.1 Contraintes](#)
 - [7.2 Hypothèses](#)
- [8. Équipe et responsabilités](#)
 - [8.1 Équipe](#)
 - [8.2 Rôles](#)
- [9. Budget et ressources](#)
 - [9.1 Budget](#)
 - [9.2 Ressources matérielles](#)

- 10. [Critères d'acceptation](#)
 - 10.1 [Fonctionnalités](#)
 - 10.2 [Performance](#)
 - 10.3 [Sécurité](#)
 - 11. [Annexes](#)
 - 11.1 [Maquettes](#)
 - 11.2 [Documentation technique](#)
 - 11.3 [Références](#)
-

1. Présentation du projet

1.1 Contexte

Loc-Manager est une application web conçue pour simplifier la gestion d'une agence de location de véhicules. Elle vise à répondre aux besoins des petites et moyennes agences de location en offrant une solution numérique pour gérer les clients, les véhicules, les réservations et les contrats de location. L'application permettra d'améliorer l'efficacité opérationnelle, de réduire les erreurs manuelles et d'offrir une meilleure expérience client.

1.2 Objectifs

- Automatiser la gestion des réservations et des contrats de location.
- Fournir une interface intuitive pour les employés de l'agence.
- Permettre le suivi en temps réel de l'état des véhicules (disponible, loué, en maintenance).
- Gérer les informations des clients et des employés.
- Générer des rapports de location pour analyser les performances de l'agence.
- Assurer une expérience utilisateur fluide et sécurisée.

1.3 Public cible

- **Responsables d'agence** : Pour gérer les opérations quotidiennes (réservations, contrats, véhicules).
 - **Employés de l'agence** : Pour enregistrer les locations et suivre les véhicules.
 - **Clients** : Pour consulter les véhicules disponibles et effectuer des réservations (si une interface client est prévue).
-

2. Spécifications fonctionnelles

2.1 Fonctionnalités principales

2.1.1 Authentification

- **Connexion** : Les employés se connectent avec un email et un mot de passe.
- **Rôles** :
 - **Administrateur** : Accès complet (gestion des employés, véhicules, rapports).

- **Employé** : Accès limité (gestion des réservations et contrats).
- **Déconnexion** : Option pour se déconnecter.

2.1.2 Gestion des véhicules

- **Ajout/Modification/Suppression** : CRUD (Create, Read, Update, Delete) pour les véhicules.
 - Informations : marque, modèle, immatriculation, année, état (disponible, loué, en maintenance), tarif journalier.
- **Suivi de l'état** : Mise à jour de l'état du véhicule (ex. "loué" lorsqu'une réservation est confirmée).
- **Historique** : Suivi des locations passées pour chaque véhicule.

2.1.3 Gestion des clients

- **Ajout/Modification/Suppression** : CRUD pour les clients.
 - Informations : nom, prénom, email, numéro de téléphone, adresse, permis de conduire (numéro et date d'expiration).
- **Historique** : Liste des locations effectuées par chaque client.

2.1.4 Gestion des réservations

- **Création** : Enregistrement d'une réservation avec :
 - Client associé.
 - Véhicule sélectionné.
 - Dates de début et de fin.
 - Statut (en attente, confirmée, annulée).
- **Modification/Annulation** : Possibilité de modifier ou annuler une réservation.
- **Validation** : Vérification de la disponibilité du véhicule avant confirmation.

2.1.5 Gestion des contrats

- **Génération** : Création automatique d'un contrat de location à partir d'une réservation confirmée.
 - Informations : client, véhicule, dates, coût total, conditions générales.
- **Signature** : Option pour marquer le contrat comme signé (manuellement ou via signature électronique si implémentée).
- **Archivage** : Stockage des contrats terminés.

2.1.6 Gestion des employés

- **Ajout/Modification/Suppression** : CRUD pour les employés (administrateur uniquement).
 - Informations : nom, prénom, email, rôle, mot de passe.
- **Historique** : Suivi des actions effectuées par chaque employé (ex. création d'une réservation).

2.1.7 Rapports

- **Rapports de location** : Nombre de locations par période, revenus générés, véhicules les plus loués.
- **Exportation** : Possibilité d'exporter les rapports au format PDF ou CSV.

2.2 Fonctionnalités secondaires

- **Notifications** : Alertes pour les réservations à venir ou les véhicules en retard.
 - **Tableau de bord** : Vue d'ensemble avec les statistiques clés (nombre de véhicules disponibles, réservations en cours, revenus).
 - **Design responsive** : Interface adaptée aux écrans desktop et tablette.
-

3. Spécifications techniques

3.1 Architecture

- **Frontend** : React avec Vite pour une interface utilisateur rapide et moderne.
- **Backend** : Firebase Functions pour les opérations serverless.
- **Base de données** : Firebase Firestore pour stocker les données.
- **Authentification** : Firebase Authentication pour la gestion des utilisateurs.

3.2 Technologies utilisées

- **React** : Framework frontend pour l'interface utilisateur.
- **Vite** : Outil de build pour un développement rapide et des builds optimisés.
- **Firebase** :
 - **Authentication** : Gestion des utilisateurs (email/mot de passe).
 - **Firestore** : Base de données NoSQL pour les données structurées.
 - **Functions** : Backend serverless pour les opérations sécurisées.
- **Tailwind CSS** : Framework CSS pour un design responsive.
- **Framer Motion** : Animations fluides (optionnel, pour les transitions).

3.3 Structure des données (Firestore)

Collection `vehicles`

- `vehicleId` (ID) : Identifiant unique du véhicule.
- `brand` (string) : Marque (ex. "Toyota").
- `model` (string) : Modèle (ex. "Corolla").
- `licensePlate` (string) : Immatriculation.
- `year` (number) : Année de fabrication.
- `status` (string) : État ("available", "rented", "maintenance").
- `dailyRate` (number) : Tarif journalier (en euros).

- `createdAt` (timestamp) : Date d'ajout.

Collection `clients`

- `clientId` (ID) : Identifiant unique du client.
- `firstName` (string) : Prénom.
- `lastName` (string) : Nom.
- `email` (string) : Adresse email.
- `phone` (string) : Numéro de téléphone.
- `address` (string) : Adresse.
- `licenseNumber` (string) : Numéro de permis de conduire.
- `licenseExpiration` (timestamp) : Date d'expiration du permis.
- `createdAt` (timestamp) : Date d'ajout.

Collection `reservations`

- `reservationId` (ID) : Identifiant unique de la réservation.
- `clientId` (string) : ID du client.
- `vehicleId` (string) : ID du véhicule.
- `startDate` (timestamp) : Date de début.
- `endDate` (timestamp) : Date de fin.
- `status` (string) : Statut ("pending", "confirmed", "cancelled").
- `createdAt` (timestamp) : Date de création.

Collection `contracts`

- `contractId` (ID) : Identifiant unique du contrat.
- `reservationId` (string) : ID de la réservation associée.
- `clientId` (string) : ID du client.
- `vehicleId` (string) : ID du véhicule.
- `startDate` (timestamp) : Date de début.
- `endDate` (timestamp) : Date de fin.
- `totalCost` (number) : Coût total (en euros).
- `signed` (boolean) : Contrat signé (true/false).
- `createdAt` (timestamp) : Date de création.

Collection `employees`

- `employeeId` (ID) : Identifiant unique de l'employé (UID de Firebase Auth).
- `firstName` (string) : Prénom.
- `lastName` (string) : Nom.

- `email` (string) : Adresse email.
- `role` (string) : Rôle ("admin", "employee").
- `createdAt` (timestamp) : Date d'ajout.

3.4 Contraintes techniques

- **Sécurité :**
 - Authentification obligatoire pour accéder à l'application.
 - Les opérations Firestore (lecture/écriture) doivent être effectuées via Firebase Functions.
 - **Performance :**
 - Chargement des données (véhicules, réservations) en moins de 2 secondes.
 - Pagination pour les listes longues (ex. rapports, historique).
 - **Compatibilité :**
 - Navigateurs supportés : Chrome, Firefox, Safari, Edge (dernières versions).
 - Responsive design pour desktop et tablette.
-

4. Exigences non fonctionnelles

4.1 Ergonomie

- Interface intuitive avec une navigation claire.
- Design moderne et professionnel.
- Temps de chargement minimal pour les listes et formulaires.

4.2 Sécurité

- Authentification sécurisée via Firebase Authentication.
- Protection des données client (Firestore).
- Validation des entrées utilisateur (ex. format email, dates valides).

4.3 Disponibilité

- L'application doit être disponible 24/7, sauf pendant les mises à jour planifiées.
 - Déploiement sur Firebase Hosting pour une haute disponibilité.
-

5. Livrables

5.1 Livrables principaux

- **Code source** : Dépôt GitHub avec le code client (React) et serveur (Firebase Functions).
- **Documentation** :
 - README.md avec instructions d'installation, d'utilisation, et de déploiement.

- Documentation technique (structure des données, API des fonctions).
- **Application déployée** : URL de production (ex. `https://loc-manager.web.app`).
- **Captures d'écran** : Screenshots des principales fonctionnalités (tableau de bord, gestion des véhicules, réservations).

5.2 Livrables secondaires

- **Présentation** : Diapositives pour une éventuelle présentation (introduction, fonctionnalités, défis, démo).
 - **Vidéo de démonstration** : Vidéo montrant l'utilisation de l'application (optionnel).
-

6. Planning et jalons

6.1 Phases du projet

1. Phase 1 : Analyse et conception (1 semaine)

- Rédaction du cahier des charges.
- Conception de l'architecture (Firestore, Firebase Functions).
- Maquettes de l'interface utilisateur (optionnel).

2. Phase 2 : Développement (4-6 semaines)

- Configuration de Firebase (Auth, Firestore, Functions).
- Développement de l'authentification et des rôles.
- Développement de la gestion des véhicules, clients, réservations, et contrats.
- Développement des rapports et du tableau de bord.

3. Phase 3 : Tests et déploiement (1-2 semaines)

- Tests unitaires et fonctionnels.
- Tests de performance et de compatibilité.
- Déploiement sur Firebase Hosting et Functions.
- Corrections de bugs.

4. Phase 4 : Documentation (1 semaine)

- Rédaction de la documentation (README, technique).
- Préparation des captures d'écran.

6.2 Jalons

- **Semaine 1** : Cahier des charges validé.
 - **Semaine 3** : Authentification et gestion des véhicules terminées.
 - **Semaine 5** : Gestion des réservations et contrats implémentée.
 - **Semaine 7** : Rapports et tableau de bord terminés.
 - **Semaine 8** : Tests terminés, application déployée.
-

7. Contraintes et hypothèses

7.1 Contraintes

- **Budget** : Utilisation de Firebase (plan gratuit ou payant selon les besoins).
- **Délais** : Projet à finaliser dans un délai de 8 semaines.
- **Ressources** : Équipe de 1 à 3 développeurs.

7.2 Hypothèses

- Les employés ont un accès internet stable.
- Firebase reste disponible et performant pendant toute la durée du projet.
- Les données des clients et véhicules sont fournies par l'agence.

8. Équipe et responsabilités

8.1 Équipe

- **Développeur principal** : [Votre nom] - Responsable du développement frontend, backend, et déploiement.
- **[Membre 2]** : [Rôle] - [Responsabilités].
- **[Membre 3]** : [Rôle] - [Responsabilités].

8.2 Rôles

- **Développeur frontend** : Implémentation de l'interface utilisateur.
- **Développeur backend** : Configuration de Firebase Functions et gestion des données.
- **Testeur** : Validation des fonctionnalités et tests de compatibilité.

9. Budget et ressources

9.1 Budget

- **Firebase** : Plan gratuit (Spark) pour le développement, passage au plan Blaze si nécessaire.
- **Hébergement** : Firebase Hosting (gratuit pour les besoins de base).
- **Outils** : VS Code (gratuit), GitHub (gratuit), Firebase CLI (gratuit).

9.2 Ressources matérielles

- Ordinateur avec Node.js 18+ installé.
 - Connexion internet pour le développement et le déploiement.
-

10. Critères d'acceptation

10.1 Fonctionnalités

- Les employés peuvent se connecter et gérer les véhicules, clients, réservations, et contrats.
- Les rapports sont générés et exportables.
- L'application est responsive et fonctionne sur desktop et tablette.

10.2 Performance

- Temps de chargement des listes < 2 secondes (sous bonne connexion).
- Pas de crash ou erreur bloquante lors de l'utilisation.

10.3 Sécurité

- Les données sont protégées (Firestore).
- Les opérations nécessitent une authentification.

11. Annexes

11.1 Maquettes (optionnel)

- Maquettes du tableau de bord, des formulaires, et des listes (à fournir si disponibles).

11.2 Documentation technique

- Structure des collections Firestore (décrite dans la section 3.3).
- API des Firebase Functions (à documenter dans le README).

11.3 Références

- Documentation React : <https://react.dev/>
 - Documentation Vite : <https://vite.dev/>
 - Documentation Firebase : <https://firebase.google.com/docs>
-